mcpro (runner patch) — guide

Revision: 1 **Last modified:** 2026-09-26T14:20:00Z

Source: `constitution/scripts/codegraph/runner_patches/mcpro1.py` (161 lines). A **module driven by `fk_index_patch.py`**, not a command. Citations are `mcpro1.py:<line>` unless another file is named. Machinery guide: fk_index_patch.

Overview

`mcpro1` builds a **READ-ONLY CodeGraph runner for MCP serving**. It is never used for `init`, `index` or `sync`. It exists because stock 1.6.0 `codegraph serve --mcp` is not read-only (docstring lines 3-14):

1. `mcp/engine.js` runs `catchUpSync()` (`cg.sync()`, a write path) at startup and starts a file watcher; only the watcher has an env kill switch.
2. Every `DatabaseConnection.open()` runs migrations, `healBulkNodeLoad`, `healBulkSecondaryIndexes` (DDL: recreates deliberately dropped bulk indexes) and `healOversizedWal`.
3. `FileLock.acquire()` treats a lock older than 2 minutes as stale whatever the holder PID, so an MCP server started next to a live bulk indexer can steal its lock and sync concurrently.
4. The default mode spawns a detached daemon per project (pid/log/socket files plus up to 16 workers that each run the heal- writing open).

The patch makes the runner **incapable** of mutating a project, unconditionally (no environment gate, so a missing or typoed variable cannot re-enable a writer; lines 16-17). Eight anchors, five files (lines 59-123, 144-161):

Anchor	File	Effect
`DatabaseConnection.open(dbPath)`	`lib/dist/db/index.js`	Opens with `createDatabase(dbPath, {readOnly: true})`, `PRAGMA query_only=ON`, no migration, no heal; throws if the database is missing or its schema version is older than this build's (never migrates); the stock body is kept, unreachable, as `helixLegacyOpen` (lines 60-83)
`DatabaseConnection.initialize(dbPath)`	same	Throws: a read-only runner never creates a database (lines 84-89)
`FileLock.acquire()`	`lib/dist/utils.js`	Prints a notice and throws; no lock is ever taken and no lock file is unlinked (lines 91-99)
`FileLock.release()`	same	No-op, never unlinks (lines 100-105)
`MCPEngine.startWatching()`	`lib/dist/mcp/engine.js`	No-op plus one stderr notice containing `read-only` (lines 106-113)
`MCPEngine.catchUpSync()`	same	No-op (lines 114-119)
`daemonOptOutSet()`	`lib/dist/mcp/index.js`	Always returns true: direct mode, no daemon, no daemon files (lines 120, 138-142)
CLI allowlist before `new commander_1.Command()`	`lib/dist/bin/codegraph.js`	Only no-args, `--version` / `-V` / `-v`, `--help` / `-h` / `help`, `status`, and `serve` with `--mcp` run; anything else prints a refusal and exits **78** before it can open, lock or delete anything (lines 121-136)

The source records a measured hazard behind the allowlist (lines 27-30): without it the CLI `index` subcommand on this runner deleted the database, because `removeDatabaseFiles` runs before `initialize()` throws. Layering (lines 32-35): the CLI allowlist is the first line, the read-only open / no-op sync / no-daemon are the operational lines for the MCP server, and the `initialize` / `FileLock` throws are second-line guards for paths that bypass the CLI.

Prerequisites

Reads `db/index.js`, `utils.js`, `mcp/engine.js`, `mcp/index.js` and `bin/codegraph.js` from the package. Python stdlib plus `runner_patches/common.py`. The package must be pristine.

Registry properties (`runner_patches/__init__.py:17-19,24-25,31-34`): mcpro1 is **OPT_IN** (never in the default set) and **EXCLUSIVE** (refused if combined with any other patch). Its runner key is therefore `<version>-mcpro1`, which identifies the read-only runner exactly. The runner cache already holds one, `~/.cache/helix/codegraph_runner/1.6.0-mcpro1` (directory listing seen 2026-09-26; contents not opened).

Usage

Only through `fk_index_patch.py`; in practice through `codegraph_mcp.sh`, which builds and verifies it with `--patches mcpro1 --print-bin` (`codegraph_mcp.sh:58`, see codegraph_mcp):

```
python3 scripts/codegraph/fk_index_patch.py --patches mcpro1 --dst <scratch dir>
```

- `NOT RUN` in that form (copies the 282 MB package). Read-only planning was run: `VERIFIED (ran: fk_index_patch.plan_all(<stock 1.6.0>, ["mcpro1"]); saw: APPLIED, 8 anchors, 5 files, detail mode "unconditional read-only (no env gate)", exclusive true, opt_in true, daemon "never", watcher "never", catch_up_sync "never")`.

- `VERIFIED (ran: fk_index_patch.py --patches mcpro1,fkidx1 machinery via runner_patches.load; saw: Refuse "patch 'mcpro1' is exclusive ... cannot be combined with: fkidx1", exit 2)` (parent guide, Example 6).
- `VERIFIED (ran: tests/test_mcp_readonly.sh with TMPDIR set to a scratch directory; saw: SUMMARY cases=13 failed=0, exit 0)`. This builds the `--patches mcpro1` runner in its own workdir and exercises it on fixture projects only.

3 2 5

What the test proves (`tests/test_mcp_readonly.sh` , header lines –)

Each check that claims "nothing changed" also runs the same instrument against the **stock** runner as a control, so a green result is shown able to see a change:

R1 builder exit 0, runner present, receipt lists the anchors; R2 the RO runner answers `tools/call` on a drifted fixture; R3 every descriptor on `codegraph.db` is `O_RDONLY` (stock: read-write); R4 logical DB hash and main file unchanged after a session on a drifted tree (stock changes); R5 a dropped bulk index stays dropped (stock recreates it); R6 no watcher/auto-sync even without `CODEGRAPH_NO_WATCH` (stock reports active); R7 a planted stale lock file is untouched (stock removes it); R8 builder fail-closed on shape mismatch: exit 2 and no runner directory; R9 the RO runner refuses `codegraph sync` (exit 78; database and planted lock untouched); R10 it refuses `codegraph index` ; R11 no detached daemon even with `CODEGRAPH_NO_DAEMON=0` (stock leaves `daemon.log`); R12 node-level: `FileLock.acquire` throws and never unlinks a stale lock; R13 registry: registered, selectable, not in the default set, exclusive with writer patches. The test unsets ambient `CODEGRAPH_*` env so the instrument cannot be steered from outside (`test_mcp_readonly.sh:21`). `tests/test_mcp_readonly_mutations.sh` applies single-edit mutants (table in `test_mcp_readonly_mutate.py`).

Behaviour (source-traced)

`plan(view)` (lines 144-161): Refuse `mcpro1: <file> is ALREADY patched` if the marker `helix-mcpro1` occurs in any of the five files (lines 145-147); then eight `replace_once` edits, each failing closed (exit 2, no runner) if its anchor is not found exactly once. There is no NOT_NEEDED path. Because it edits `db/index.js` and `utils.js`, the same files touched by `fkidx1` and `lockfix1`, it must never be combined with them (enforced by EXCLUSIVE).

Edge cases

- Read-only runner asked to `init` / `index` / `sync` / `uninit` / `unlock` / `upgrade`: refuses with exit 78 (R9, R10 VERIFIED above for `sync` and `index`; other subcommands are refused by the same allowlist logic, lines 125-135, not individually run).
- Database with an older schema than the runner's build: throws `helix-mcpro1: schema vN is older than vM; a read-only runner cannot migrate` (lines 60-83; source-traced, not run).
- A project whose index is out of date: the read-only server answers from what is on disk and never catches up (R4/R6).
- Anything other than `serve --mcp`, `status`, `--version`, `--help` fails by design; for indexing use the writer runner through codegraph_safe.

Related scripts

fk_index_patch, codegraph_mcp (builds/verifies this runner and serves MCP), codegraph_mcp_preflight, codegraph_mcp_probe, siblings fkidx1, lockfix1, datafrag1, resolve1, resolve2.

Last verified

2026-09-26: `tests/test_mcp_readonly.sh` 13/13 PASS with a scratch `TMPDIR`; real runner cache listing unchanged. `tests/test_mcp_readonly_mutations.sh`: mutants=10 killed=10 survivors=0.